1/1/26 → Graph Theory.

Graph Theory: vizuvalization of Relations.

GT: $R : X \times Y \{(x,y) / (\text{function} / \text{condition})\}$ x, y – Non empty sets.

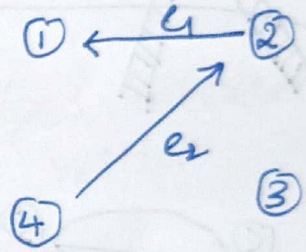
2/1/26

S1: $A = \{1,2,3,4\}$ set of $n \in \mathbb{N} \ni n > 5$

S2: $A \times A = \{ (1,1)\ (1,2)\ (1,3)\ (1,4)$
$(2,1)\ (2,2)\ (2,3)\ (2,4)$
$(3,1)\ (3,2)\ (3,3)\ (3,4)$
$(4,1)\ (4,2)\ (4,3)\ (4,4) \}$

S3: $R = \{(x,y) \in A \times A \mid x = 2y\}$

$R = \{(2,1), (4,2)\}$ (e1, e2)



Graph

$G = \langle V, E \rangle$

V = underlying set. (vertices)

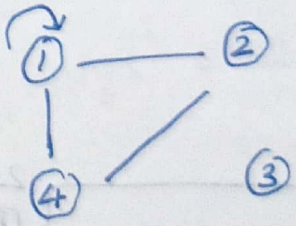
~~$E \subseteq R$~~ $E \cong R$, $E \subseteq V \times V$ (Edges)

– we will discuss about undirected graph.

Degree of vertex:

No of edges incident to a vertex

degree of 1 = 4 { with 2, with 4, In of 1, Out of 1 }



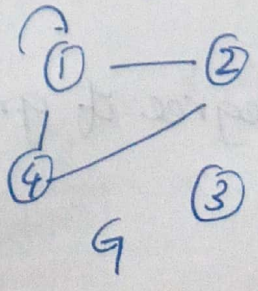
G

Adjacent Vertex:

if 2 vertexes are adjacent, there should be a edge blw them and vice versa wrt edges & vertex

– (3) is isolated vertex.

– Graph G has 2 components



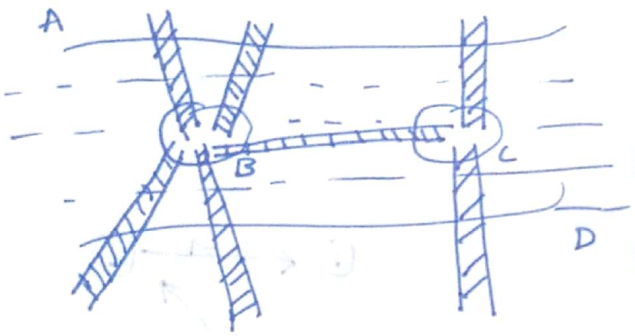
G

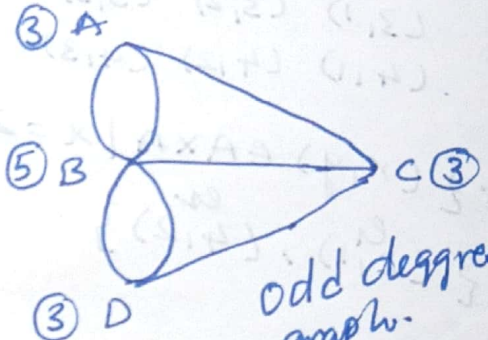
← disconn-ected Graph.

Walk:

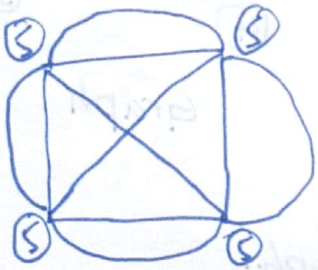
Konigs berg bridge problem:

start at Any land mass
move through all bridges
Once only and return to
Same land Mass.





Odd degree graph.



odd degree graph

Finite Graph:

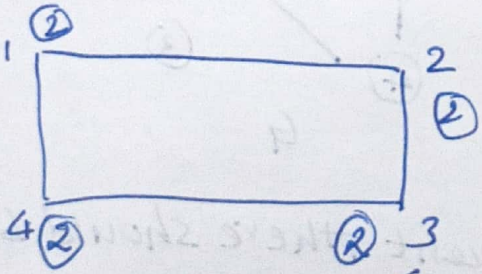
Finite vertexs & Edges.

Finite vertexes
Infinite Edges X

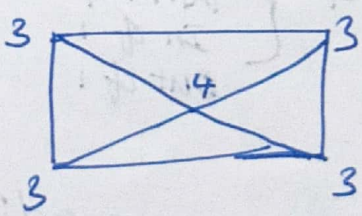
→ Only completed when the graph has all the vertecies should have even degree:

Euler Graph.

-Negation: Non- Euler Graph.



Euler Graph. ✓

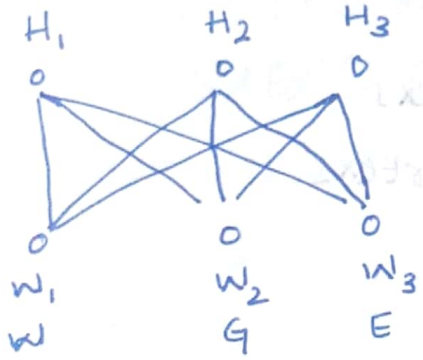


Not An Euler Graph X

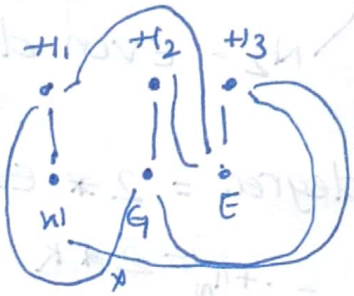
-Total degree of graph = 2x Edges (undirected)?

5/1/26

Three utilites problem:

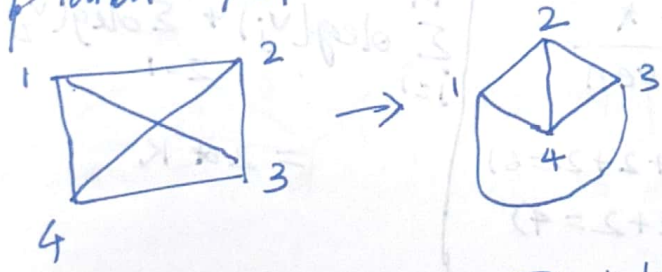


3 houses, 3 utilites (water, Gas, Elec)
NO 2 piplines should not be crossover? Is it possible / NOT?



- Its Not possible
  - Subolivision of K_5 or $K_{3,3}$.
- Non planar Graph.

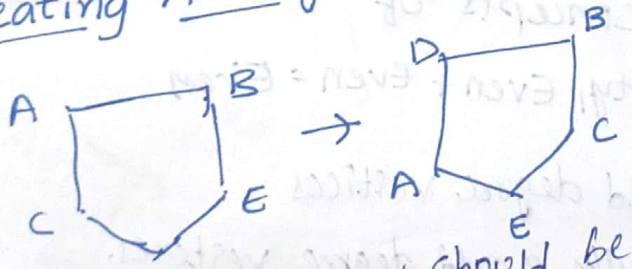
Planar Graph:



- NO two edges intersect except at vertices.

Seating Arrangement Problems:

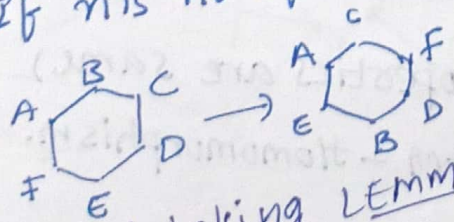
Permutation $\begin{cases} n! \\ (n-1)! \end{cases}$



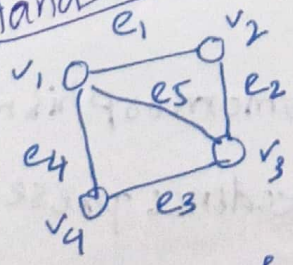
Any other? NO, 2 ways.
What if $n = 1000$?
$n = 10K$?

NO 2 persons should be adj in 2 positions

If n is no of vertices. n / Odd $\frac{n-1}{2}$ ways $5 \to 2$ ways
Even $\frac{n-2}{2}$ ways $6 \to 2$ ways.



Handshaking LEMMA



Label edges & vertexs

	degree	
v_1	3	$e_1 e_5 e_4$
v_2	2	$e_1 e_2$
v_3	3	$e_2 e_5 e_3$
v_4	2	$e_4 e_3$
	10 = 2x	5 edges

Total degree = 2x edges

degree - counting NO of edges.

v_1, v_3 - odd degree vertex

v_2, v_4 - even degree vertex

→ The No of 'odd' degree vertexes in a graph is 'Even'

Proof:

A Graph with N vertices, K Edges.

N vertices { N_1 – odd degree vertexs
N_2 – Even degree vertexs.

Total degree = 2 * Edges.

$n_1 + n_2 + n_3 + \cdots + n_N = 2 * K$

(odd deg) + (Ev deg) = 2 * K

Even

LHS only Even when No of these are Even

- 3+3 = 6 ✓
- 3+3+3 = 9 ✗

Even
- odd (2+2+2=6)
- Even (2+2=4)

$$\sum_{i=1}^{n} \deg(v_i) = 2 * K$$

$$\sum_{j=1}^{M} \text{odeg}(v_j) + \sum_{z=1}^{N-M} \text{odeg}(v_z)$$

$= 2 * K$

From the concepts of Associativity, Even + Even = Even

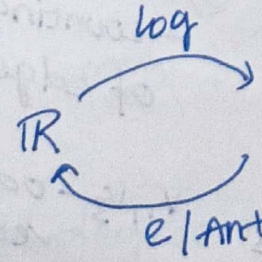
Euler circuit exists ⇔ 0 odd degree vertices

Euler path exists ⇔ exactly 2 odd degree vertexes.

Isomorphism b/w 2 Graphs:

$\mathbb{R}$ isomorphic to (1,2) (Size & properties are same)

If sizes differ – Homomorphism.



⇒ Isomorphism
- bijective Homomorphism.
- bijective + Incedence preserving

Sampling data.

-2 Graphs are said to Isomorphic

$G_1 \langle V_1, E_1 \rangle$ & $G_2 \langle V_2, E_2 \rangle$.

If one-one & onto relationship b/w V_1 & V_2 & E_1 & E_2 is provided to preserve Incident Relationship.

$|V_1| = |V_2|$

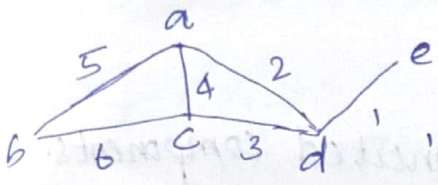
$|E_1| = |E_2|$

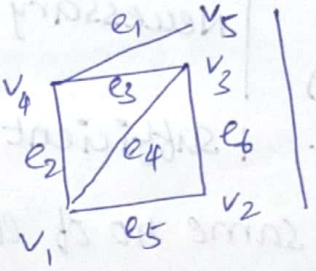
- same structure
- Labels may differ
- shape doesn't matter

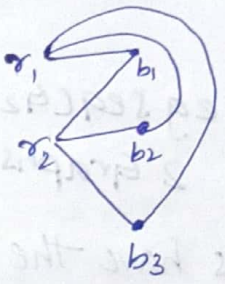
def: $G_1 = (V_1, E_1)$, $G_2 = (V_2, E_2)$

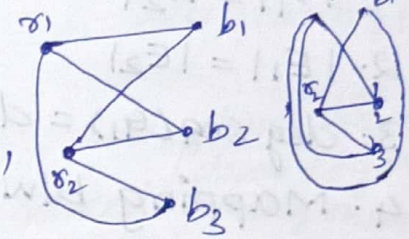
$f: V_1 \to V_2$

$\{u, v\} \in E_1 \Leftrightarrow \{f(u), f(v)\} \in E_2$









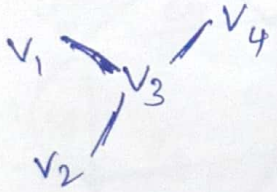
Note

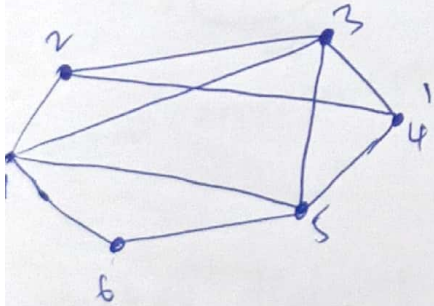
$V_1 - V_2 - V_3 - V_4$] same NO of edges, same NO of vertices

deg seq = $\langle 1, 2, 2, 1 \rangle$

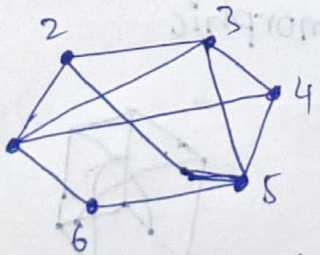
$= \langle 1, 1, 3, 1 \rangle$

Not isomorphic





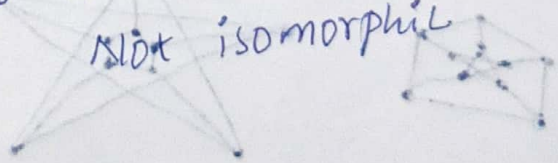
$\langle 4, 3, 4, 3, 4, 2 \rangle$



$\langle 4, 3, 4, 3, 4, 2 \rangle$

] same Edges, same vertices, same degree sequence

Not isomorphic



8/11/26

Isomorphism B/w Graphs

$G_1 = \langle V_1, E_1 \rangle$ $G_2 = \langle V_2, E_2 \rangle$

1-1 correspondence b/w vertices of G_1 & G_2 $|G_1| = |G_2|$
Edges of E_1 & E_2 $|E_1| = |E_2|$

Incedence relationship is preserved.

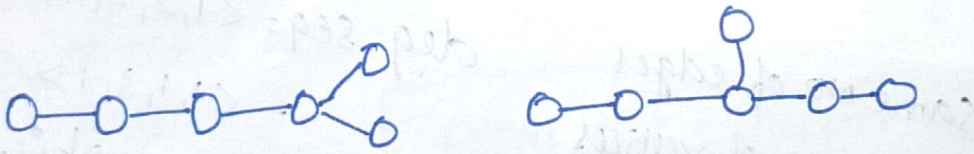
Steps to check it

1. $|V_1| = |V_2|$
2. $|E_1| = |E_2|$
3. deg seq(G_1) = deg seq(G_2)
4. Mapping b/w 2 Graphs.

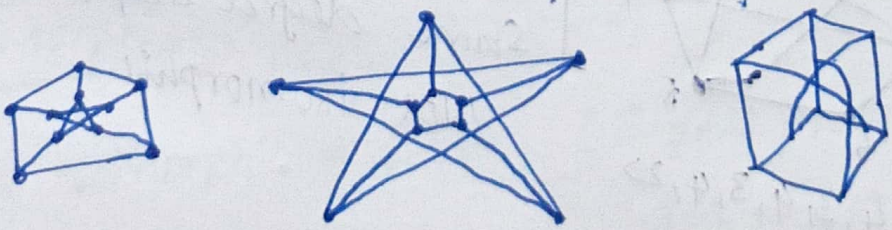
Necessary | Sufficient.

* Adjacency preservation
if u, v are adj in G_1, then f(u), f(v) must be adj in G_2

- Isomorphic graphs have the same no of connected components.



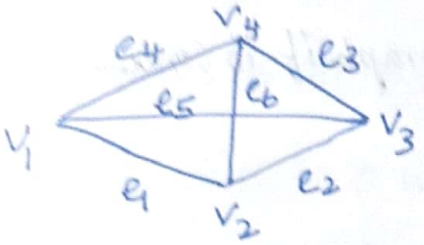
$|V_1| = |V_2|$
$|E_1| = |E_2|$
deg are equal
But It's NOT isomorphic



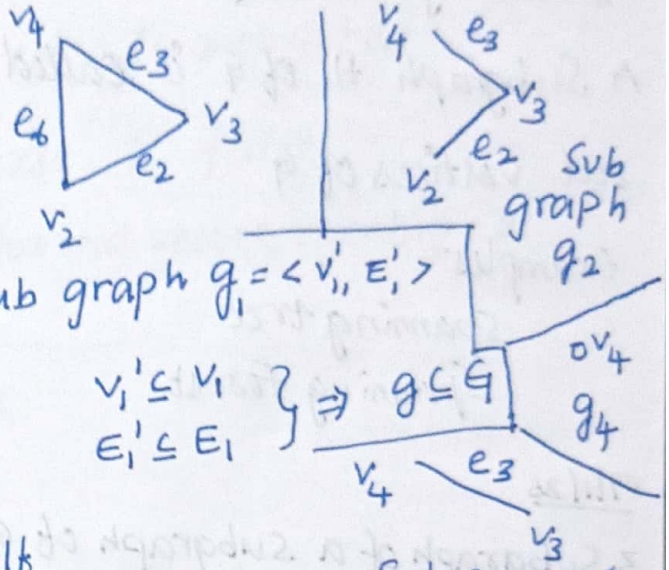
Isomorphic /b/w we can get clear idea in any one of the graph easily.

- Isomorphic graphs are one & same.
Isomorphic graphs are structurally Identical.

Sub-Graph



Graph $G = \langle V_1, E_1 \rangle$



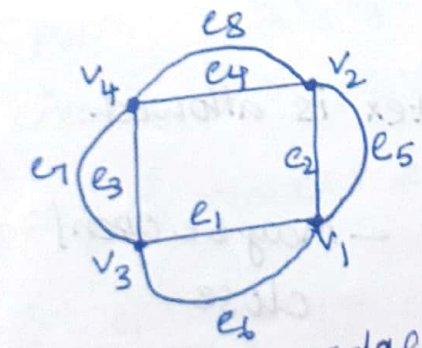
$V_1' \subseteq V_1$
$E_1' \subseteq E_1$
$\Rightarrow g \subseteq G$

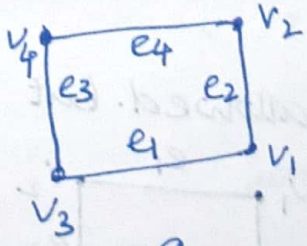
1. Every Graph is subgraph of itself
2. Null Graph is subgraph of Every Graph.

$\dagger \langle V_1, \phi \rangle$
All vertices, NO edges.

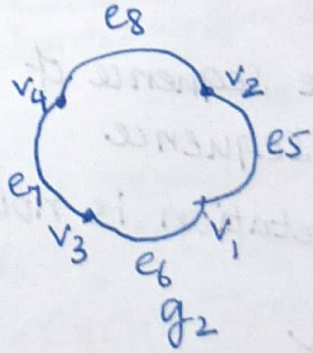
2 types:
1. vertex-Induced — Choose vertex, draw edges among them
2. Edges Induced — Choose Edges, include End points

Edge disjoint Sub Graph





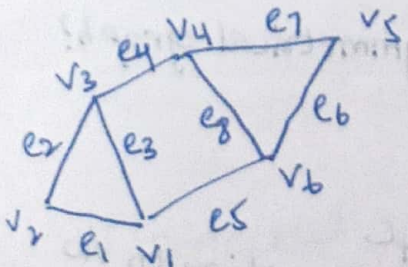
g_1

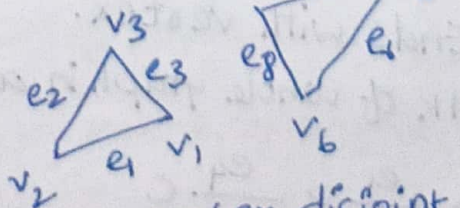


g_2

- NO Common Edges
- g_1 & g_2 are edge disjoint Sub graph.

Vertex disjoint Sub graph





Vertex disjoint Subgraph

- NO common vertex
- Every vertex disjoint Subgraph is Edge disjoint Subgraph but converse is not

Spanning Subgraph.

A Subgraph H of G is called a spanning subgraph if it contians all vertices of G

Examples:
- Spanning tree
- Spanning forest

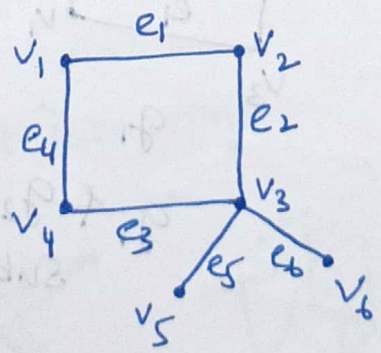
9/1/26

3. Subgraph of a subgraph of G is also subgraph of G

Types:
1. Walk
2. Path
3. Circuit

Walk:

2. Alternate sequence of vertices & Edges.
1. A Finite sequence
3. Edge repetation is not allowed. but vertex is allowed.



- may be open/ close.

$v_4 e_4 v_1$ ✓

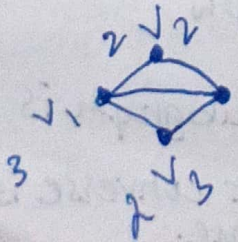
$v_4 e_4 v_1 e_1 v_2 e_2 v_3 e_3 v_4$ ✓

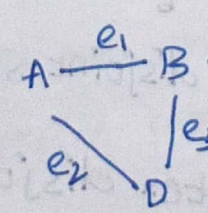
$v_2 e_2 v_3 e_3 v_4$ ✓

$v_6 e_6 v_3 e_2 v_2 e_1 v_1 e_4 e_3 v_3 e_5 v_5$ ✓

Every walk is start & Ends with vertex.

Q) Can we complete walk of whole graph in asymmetrical graph?





$A e_2 D e_3 B e_4 C$

→ so, Symmetric has nothing to do but degree matters.

Path:

1. Walk + No vertex repetation. — open walk

No vertex is repeated, hence no edge is repeated.

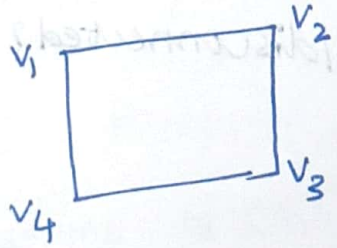
Every path is a walk but converse is false

- deg of vertex in path $\in \{1,2\}$ 1 for end vertex, remaing = 2.
- length of path = No of edges
- A self loop cannot be a path.

Circuit

A closed path → self loop is circut

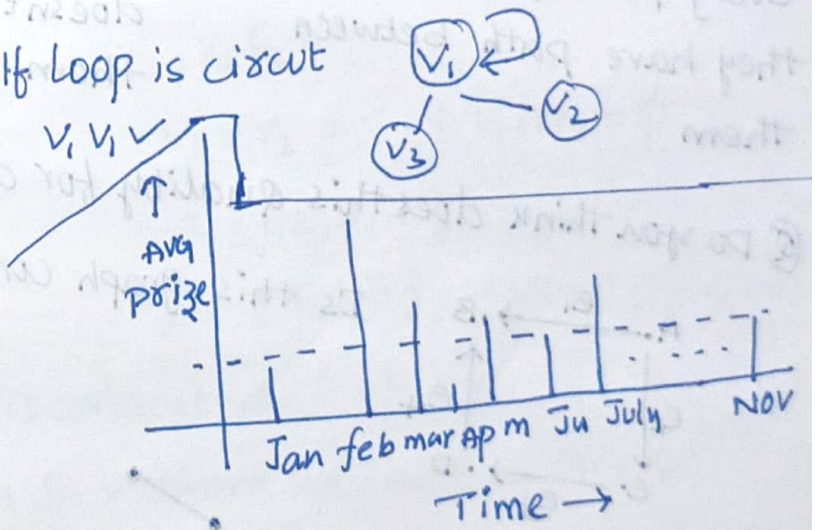
start = end.



Path: $V_1 V_2 V_3 V_4$

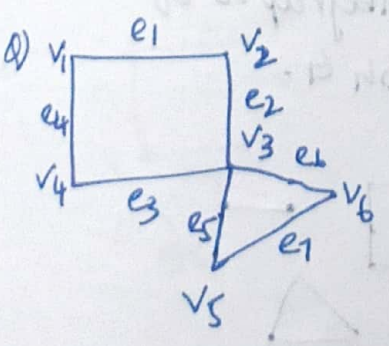
Circuit: $V_1 V_2 V_3 V_4 V_1$

deg of vertex = 2



Every 4 months AVG prize is same, so its cyclic.

Cyclic Trend.

Q) 

No of circuts in Given Graph ?

$V_1 V_2 V_3 V_4 V_1$ ✓

$V_3 V_5 V_6 V_3$ ✓

$V_3 V_4 V_1 V_2 V_3 V_5 V_6 V_3$ — ?

$V_4 V_1 V_2 V_3 V_6 V_5 V_3 V_4$ — ?

} closed walks.

- In connected graphs, closed walk contain circuit as subgraph.
- A connected graph has no circut at all. — Trees.

12/01/26

Types of Subgraph:

1. Walk
2. Path : Non-intersecting walk.
3. Circut: Non-intersecting closed walk.

Graph

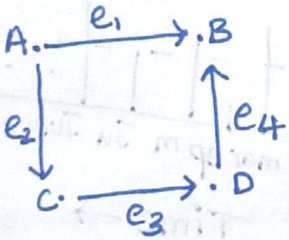
Connected

Every pair of vertices they have path between them

Disconnected.

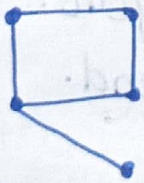
Atleast a pair of vertices doesn't have path between them.

Q) Do you think does this Quality for directed graph?

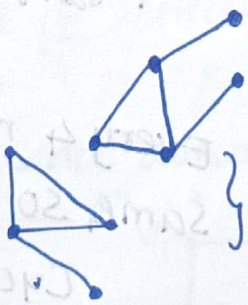


Is this graph connected/disconnected?

Ex:



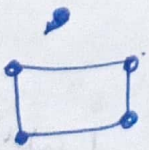
Connected



disconnected

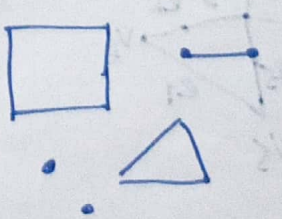
2 components

Leach component is a subgraph of Graph G.

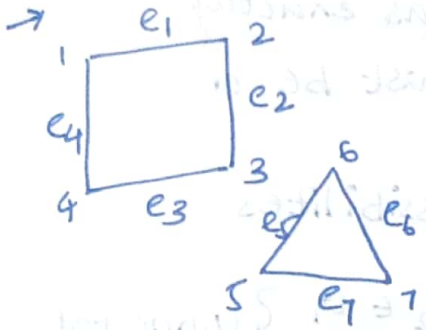


disconnected.
2 components

disconnected
3 components



5 components
disconnected.



$G = \langle V, E \rangle$

$V = \{1, 2, 3, 4, 5, 6, 7\}$

$E = \{e_1, e_2, e_3, e_4, e_5, e_6, e_7\}$

→ converse is also true for this statment →

P ≡ V is divided into 2 disjoint subsets such that there is no path between vertices of V_1 and vertices of V_2

q ≡

→ If G is disconnected

$V_1 \cup V_2 = V$

$V_1 \cap V_2 = \phi$.

$V_1, V_2 \subseteq V$

$V_1 \cup V_2 = V$ & $V_1 \cap V_2 = \phi$.

$V_1 = \{1, 2, 3, 4\}$

$V_2 = \{5, 6, 7\}$

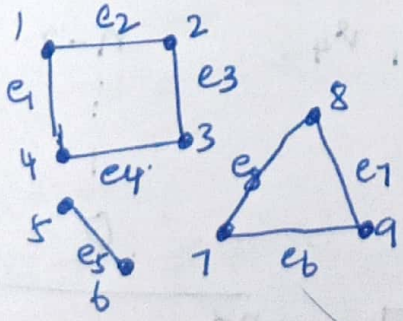
→ Assume $G \langle V, E \rangle$ is disconnected.

$a \in V_1$, collect all vertices in v there has path to a.

$V_1 = \{a, v_1, v_2, \ldots v_n\}$ $V_1 \subset V$

$V_2 = V \sim V_1$

→



$G = \langle V, E \rangle$

$V = V_1 \cup V_2 \cup V_3$

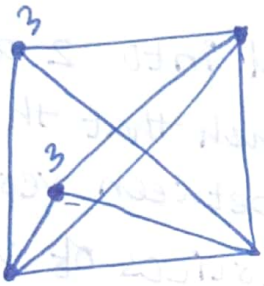
$V_1 \cap V_2 = V_2 \cap V_3 = V_3 \cap V_1 = \phi$.

$V_1, V_2, V_3 \subset V$

Generalized defination:

→If a Graph G (connected, disconnected) has exactly two odd degree vertices then there must be a path joining them.

Proof:



$G = \langle V, E \rangle$

$V = V_1 \cup V_2$

3 possibilites

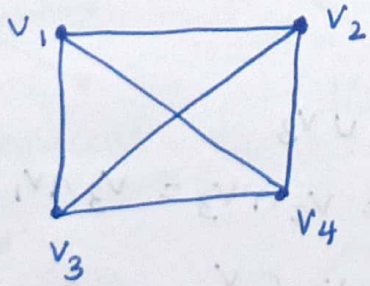
1. $v_1, v_2 \in V_1$ } connected
2. $v_1 v_2 \in V_2$
3. $v_1 \in V_1, v_2 \in V_2$ ✗ Not possible

↳from No of odd degree vertexes in a graph is even

Simple Graph.

The Max No of Edges possible for a simple graph G with n vertices is $\frac{n(n-1)}{2}$.

- Avoid self loops & parallel Edges / Multiple Edges b/w 2 vertex



for v_1 : v_2, v_3, v_4, $n-1$

v_2 : v_3, v_4 $n-2$

v_3 : v_4 1

v_4 : 0 0

v_1 : $v_2, v_3, \ldots v_n$: $n-1$

v_2 : $v_3, \ldots v_n$: $n-2$

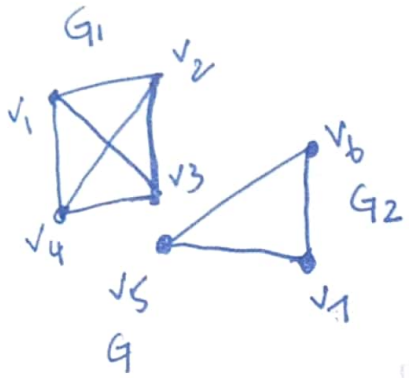
v_{n-1} : v_n 1

v_n : — 0

$$\Rightarrow \sum_{i=1}^{n-1} i \qquad \sum_{i=1}^{n-1} i = \frac{(n-1)(n-1+1)}{2} = \frac{n(n-1)}{2}$$

15/01

→ A disconnected graph G with n vertices and K components can have atmost $\frac{(n-K)(n-K+1)}{2}$ edges.



- Max No of components = n

$K \leq n$

$G = \langle V, E \rangle$, K components.

$G = G_1 \cup G_2 \cup G_3 \cdots \cup G_K$ $\quad V = V_1 \cup V_2 \cup V_3 \cdots \cup V_K$

$G = \bigcup_{i=1}^{K} G_i$

$|V_1| + |V_2| + |V_3| + \cdots |V_K| = |V| = n$

$n_1 + n_2 + n_3 + \cdots n_K = n$

$|V_i| \geq 1$ & $n_i \geq 1$

$|E_1| + |E_2| + \cdots \cdots |E_K| = ?$

$= \frac{n_1(n_1-1)}{2} + \frac{n_2(n_2-1)}{2} + \cdots \frac{n_K(n_K-1)}{2}$

$= \sum_{i=1}^{K} \frac{n_i(n_i-1)}{2}$

$= \frac{1}{2} \sum_{i=1}^{K} n_i(n_i-1)$

$= \frac{1}{2} \sum_{i=1}^{K} n_i^2 - n_i$

$= \frac{1}{2} \left[\frac{n_1(n_1+1)(2n_1+1)}{6} - \frac{n_1(n_1+1)}{2} \right]$

$= \frac{1}{2} \frac{n_1(n_1+1)}{2} \left[\frac{2n_1+1-3}{3} \right]$

$= \frac{1}{2} \left[\sum_{i=1}^{K} n_i^2 - \sum_{i=1}^{K} n_i \right]$

$= \frac{1}{2} \left[\sum_{i=1}^{K} n_i^2 \right] - \frac{n}{2}$ ——— ①

$\sum_{i=1}^{K} (n_i - 1) = n - K$

SOBS

$\left(\sum_{i=1}^{K} (n_i - 1) \right)^2 = (n-K)^2$

$\sum (n_i - 1)^2 + \text{non neg terms} = (n-K)^2$

$\sum_{i=1}^{K} n_i^2 - 2n_i + 1 + \text{non neg terms} = n^2 - 2nK + K^2$

$$\sum_{i=1}^{K} n_i^2 - 2n + K + Q = n^2 - 2nK + K^2$$

$$\sum_{i=1}^{K} n_i^2 \leq n^2 - 2nK + K^2 + 2n - K$$

$$\leq n^2 + (n-K)(2n-K)$$

from eq ①

$$\frac{1}{2}\sum_{i=1}^{K} n_i^2 - \frac{n}{2} \leq \frac{1}{2}\left[(n^2 - (2n-K)(K-1))\right] - \frac{n}{2}$$

$$\leq \frac{(n-K)(n-K+1)}{2}$$

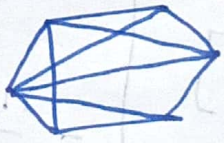
The upper bound value. but not the max.

for ex q, 7 vertices, 2 components

upper bound = $\frac{(7-2)(7-2+1)}{2}$

$= \frac{5 \times 6}{2}$

$= 5 \times 3 = 15$



(15)

Euler Graph:

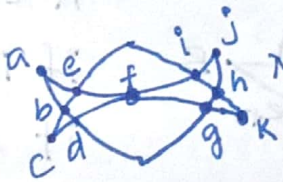
A graph G is said to be Euler graph, if it consits Euler line in it

A graph that contains of an Euler line is called Euler graph

Euler line: A closed walk in a graph contains all the edges of graph, then the walk is called Euler line.

- Euler graph is always connected

star of david



Mohamed Scimitor.

Theorm: A given connected graph G is an Euler graph if and only if all vertices of G are of even degree.

proof: Suppose that G is Euler graph, it therefore contains an Euler line

Euler line (T): $v_1 e_1 v_2 e_2 \ldots e_k v_1$

every time the walk meets a vertex v it goes through 2 new edges $\Rightarrow$ deg += 2 each time vertex v occurs.

$\rightarrow$ True for all intermediate vertex, but also for terminal vertex too, beacuse exited and entered the same vertex at beginning and end of walk

$\Rightarrow$ Terminal vertex deg += 2.

$\rightarrow$ so every vertex has even degree.

$P \Rightarrow q$ ✓

To prove sufficieny we prove, assume that all vertices of G are of even degree, Now we construct a walk starting at an arbitrary vertex v and going through the edges of G such that no edge traced more than once

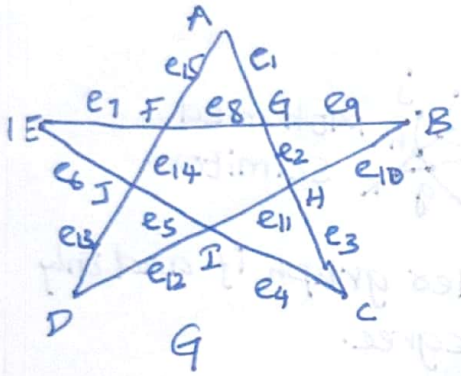
$(q \rightarrow p \Rightarrow p \Leftrightarrow q)$

Proof:

Every vertex v is even degree, so when tracing we can exit from every vertex we enter.

Tracing cannot stop at any vertex but starting one (v)

since v is also even degree we reach v when tracing comes to end.



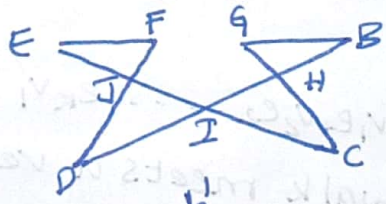
G

A closed walk'h' includes edges of G

if it contain all edges → Euler line

if not, now we remove from G all the edges in h and obtain a subgraph h'

h: A e1 G e8 F e15 A



G - h = h'

G → All even degree

h → All even degree

h' → All even degree

it touches let say a. → h' has to touch h at least once at vertex

→ Since all the vertices of h' are even degree, this walk in h' must terminate at vertex a.

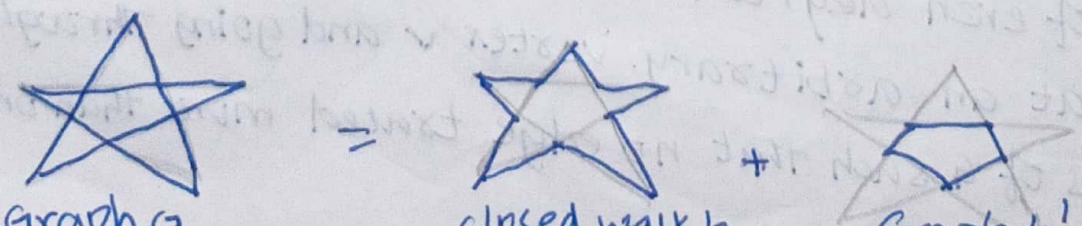
h'(walk) = G e9 B e10 H e2 G

h'(walk) + h = new walk → closed walk, starts & ends same vertex

A e1 G e9 B e10 H e2 G e8 F e15 A.

→ This process be repeated until we obtain a closed walk that traverses all the edges of G

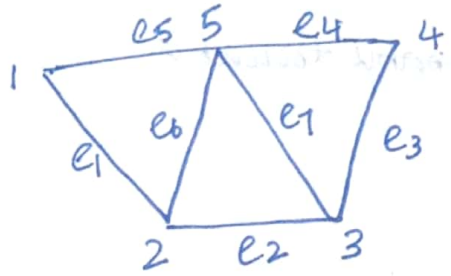
Thus G is an Euler graph.



Graph G closed walk h Graph h'

Open Euler line: Euler line but its open walk.
or
Unicursal line

Unicursal Graph: A Graph containing unicursal line.



3 even degree vertex
2 odd degree vertex

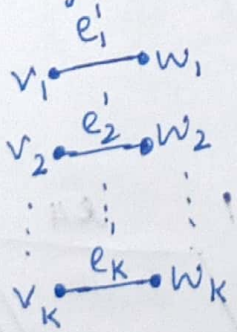
Unicursal line: $2e_1 1 e_5 5 e_6 2 e_2 3 e_7 5 e_4 4 e_3 3$.

19/01/26.

- Except 2 vertices all other vertices has even degree.
- In order to make unicursal graph into Euler graph, add a edge b/w those 2 vertices.

T: If a connected Graph G with 2K odd degree ~~vertex~~ vertices, edge then it can be decomposed into K disjoint subgraphs (which are unicussal).

P: 2K odd degree vertexs = $\{v_1 \cdots v_k, w_1, \cdots w_x\}$

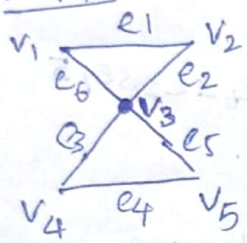


To make graph into Eules graph, we add K edges.

- In a connected graph G with 2K odd degree vertex, there exist K edge disjoint subgraphs such that they together contain all edges of G and that each is unicursal graph.

- A Euler graph is said to be Arbitrarily traceable from a vertex v, when you can travel in any way so that you end up with a Euler line.

22/11/24



It's a graph, arbitrarily ~~traable~~ tracable graph from a vertex v_3.

- Unicursal graph is subgraph of Euler graph
- A Euler graph is Supergraph of unicursal graph

Operations on Graphs.

$G_1 = \langle V_1, E_1 \rangle$

$G_2 = \langle V_2, E_2 \rangle$

$G_1 \cup G_2 = \langle V, E \rangle$

$V = V_1 \cup V_2$

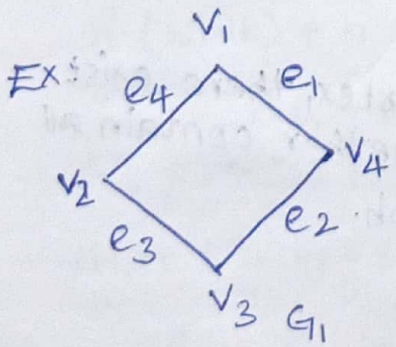
$E = E_1 \cup E_2$

$G_1 \cap G_2 = \langle V', E' \rangle$

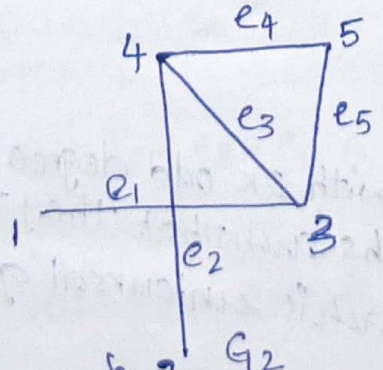
$V_1 \cap V_2 = V'$

$E_1 \cap E_2 = E'$

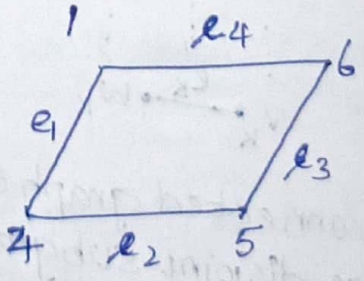
Ex:



G_1

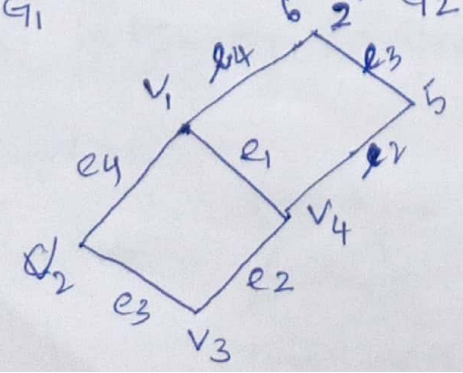


G_2

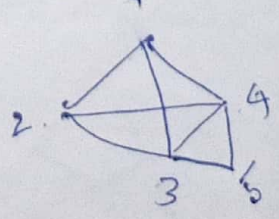


G_3

$G_1 \cup G_3$

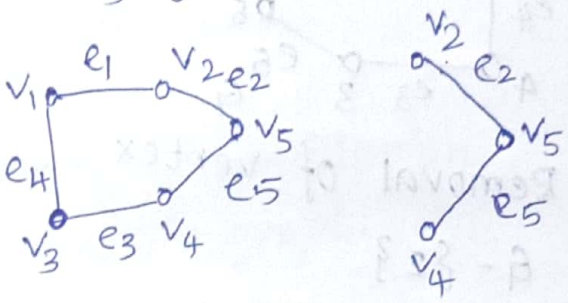


$G_1 \cup G_2$



$G_1 \cap G_3$

If g is subgraph of G

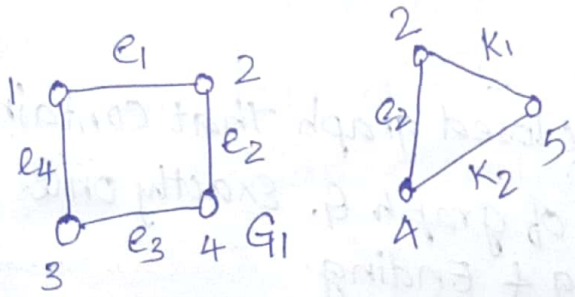


Ring Sum:

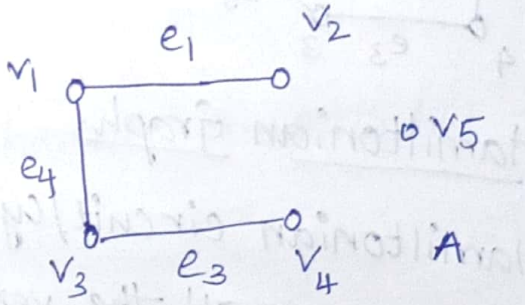
$G_1 \oplus G_2 = \langle V'', E'' \rangle$

$V'' = V_1 \cup V_2$

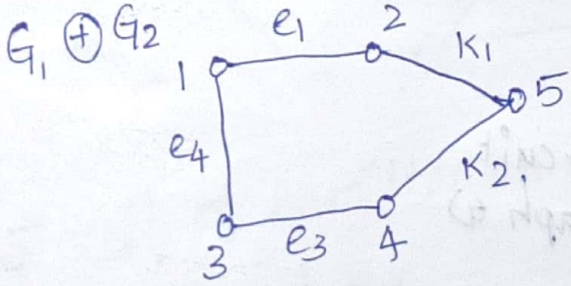
$E'' = (E_1 \cup E_2) \setminus (E_1 \cap E_2)$



$G \oplus g$.



- It is disconnected graph.
- It subgraph of G
- $A = G \oplus g = G \setminus g$
- $A \cup g = G$.
- $G \setminus g = A = G \oplus g$.

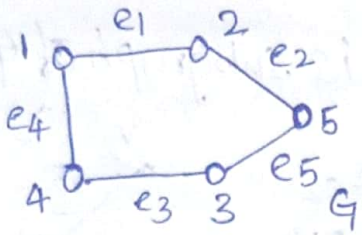


Properties:

1. $G_1 \cup G_2 = G_2 \cup G_1 = G_2 \cup G_1$
2. $G_1 \cap G_2 = G_2 \cap G_1$
3. If G_1 & G_2 are edge-disjoint graphs.

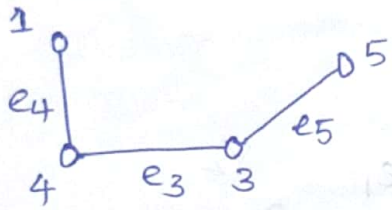
   $G_1 \oplus G_2 = G_1 \cup G_2$.

   $G_1 \cap G_2 =$ Null Graph.



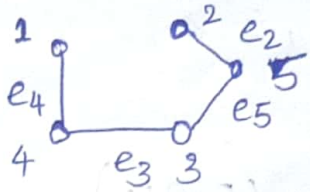
1. Removal of vertex

$G - \{2\}$



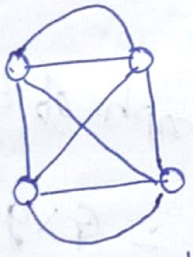
2. Removal of Edge

$G - \{e_1\}$



Hamiltonian Graph

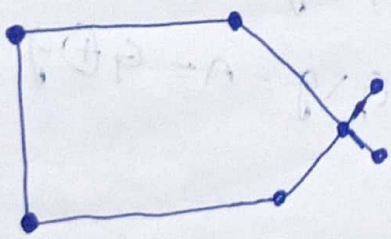
Hamiltonian circuit/cycle: A closed graph that contains all the vertices of graph G. exactly once. Except the starting & Ending.



G

Euler Graph

Hamiltonian circuit. (subgraph of Graph G)



doesn't have Hamiltonian circut.

- Not Every connected graph has a Hamiltonian circut

→ He created a problem (which is not still unsolved) what is a neccessary and sufficient condition for a connected graph G to have Hamiltonian circut?

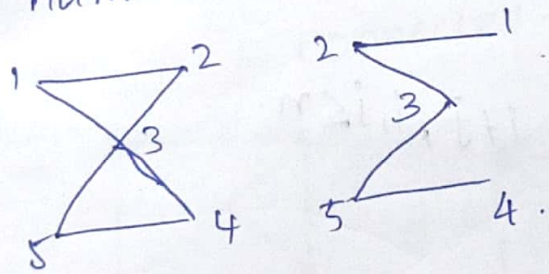
-If we remove any one edge from a Hamiltonian circut we are left out with a path, which is Hamiltonian path.

- Hamiltonian path is subgraph of Hamiltonian circut which is subgraph of another graph.

— converse is not true.

- Every graph which has hamiltonian circut has hamiltonian path in it

- There are many graphs with hamiltonian path but no hamiltonian circut



No circut.

— length of hamiltonian path of connected graph of n vertices is n-1.

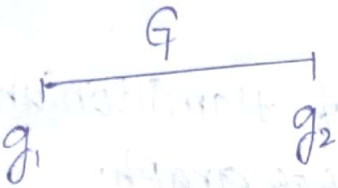
23/01/26

Eulergraph
But its not arbitarly traceable from a vertex.

Euler graph
Its arbitarly traceable from any vertex.

Euler graph:

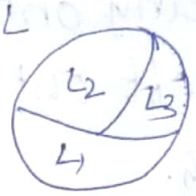
Decomposition:



A Graph G is decomposed into g_1 and g_2 then

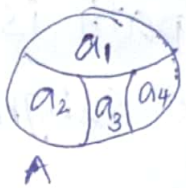
i) $g_1 \cup g_2 = G$

ii) $g_1 \cap g_2 =$ Null Graph



L_i: partition

$L = L_1 + L_2 + L_3$



a_i: partition

$A = a_1 \cup a_2 \cup a_3 \cup a_4$

$a_i \cap a_j = \phi$ for $\forall i \neq j$

- Euler graph $\Leftrightarrow$ It can be decomposed into edge disjoint Graph.

$C_1 \cup C_2 \cup C_3 \cup \ldots \cup C_n = G$

$C_i \cap C_j =$ Null Graph for $\forall\ i \neq j,\ i, j \leq n$.

- $P \rightarrow q$

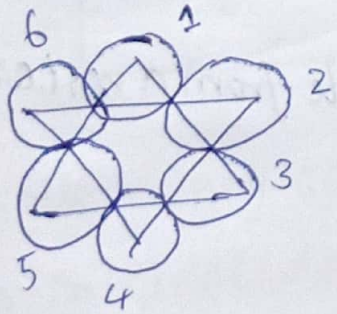
G is Euler graph $\Rightarrow$ $v_1 \in V$

$v_1 \xrightarrow{e_1} v_2 \xrightarrow{e_2} v_3 \ldots\ldots\ldots$

Traversing is stopped when

i) we reach v_1

ii) completed entirely.

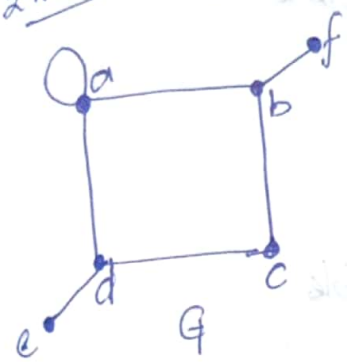


Every graph is edge disjoint.

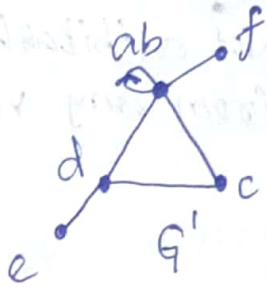
$v_1 \cup v_2 \ldots \cup v_6 = G$

- degree is always $2*n$
- So, the graph is Euler graph

29/01/26



- A Graph G is said to be "fused" fusing a & b.



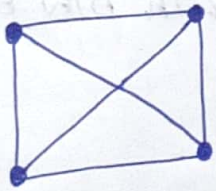
- No of Graph G
- No of vertices in G' = No of vertices - 1

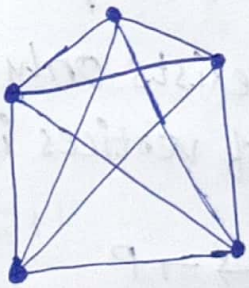
Ex: Merging 2 entries of same person in database.

Complete Graph :-

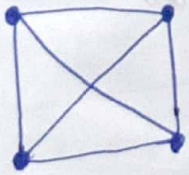
Every vertex is connected to every other vertex directly.

Ex:

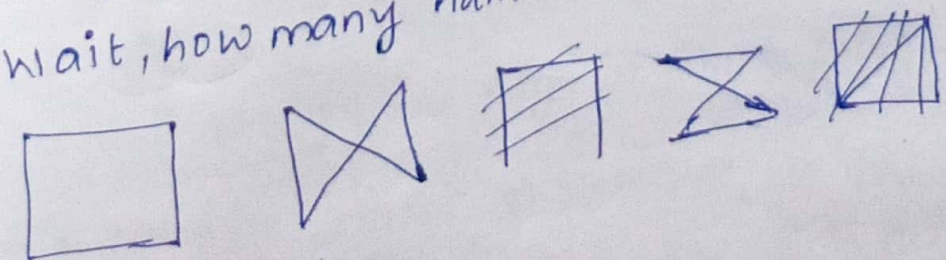




- Why does it require?



- Every Complete graph has hamiltonian circut in it.
- Wait, how many hamiltonian circuts can you build?



3 circuts.

- If a vertex is said to be arbitarly traceable. It should contain in all the circuts.

Not arbitarly traceable from any vertex.

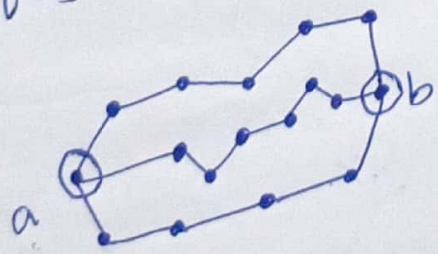
Tree:

- A type of Subgraph.
- A connected graph that has no circut in it.
- Between any 2 vertices in Graph, there should exist only one path. is called as Tree.

Theorm 1: Tree ⇒ There exists only one path b/w every pair of vertices in G.

proof: $P \rightarrow q \equiv \neg q \rightarrow \neg P$

If ~~T~~ more than one path b/w a & b.



If there exist more-than one path thesese exist a circut in it

So its not an tree.

$\neg q \rightarrow \neg P$ (proved)

So $P \rightarrow q$ also proved.

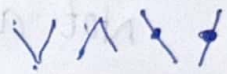
Theorem 2: A Tree with n vertices has n-1 edges. 30/01/26.

proof: By induction method.

Step 1: for 1 vertices ⇒ 0 edges.

2 vertices ⇒ 1 edge b/w them

3 vertices ⇒ 2 edges.



Step 2: Assume its true for n=k vertices:

A tree with k vertices has k-1 edges.

$T = \langle V, E \rangle$.

$|V| = k$.

$T_1 = \langle V_1, E_1 \rangle$ $T_2 = \langle V_2, E_2 \rangle$

$|V_1| = k_1$ $|V_2| = k_2$

$k = k_1 + k_2$, $k_1 < k$, $k_2 < k$.

$|E_1| = k_1 - 1$ $|E_2| = k_2 - 1$

$k_1 + k_2 - 2$

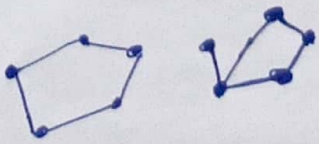
Total $k = k_1 + k_2 - 2 + 1$

$= k_1 + k_2 - 1$

13: A connected graph with 'n' vertices & 'n-1' edges is tree.

P: $P \rightarrow q \equiv \neg q \rightarrow \neg P$

In order to have a connected graph & with n vertices to have a circut in it we are required to have min of n edges



Eg: Simple polygons.

modify one edge in simple polygons.

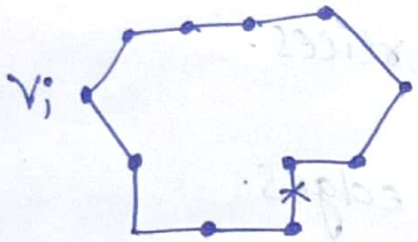
- In order to connect n vertices, we are required to have n-1 edges, but having n edges produces circut. So a connected graph with n vertices & n-1 edge

has no circuit in it, so its a tree.

proof:

Not a tree.

G has v_i, v_j which has more than 2 paths



Remove one edge to create only path

$\Rightarrow$ n-1 edges & NO circuit

Theorm 4: G is Minimally Connected. $\Leftrightarrow$ Tree

- A connected graph G is said to be "Minimally connected" if any removal of edge would lead to make Graph G is disconnected

proof: $P \Leftrightarrow q$ $\quad P \rightarrow q$, $q \rightarrow P$.

Theorm 5:

A Graph G with n vertices, n-1 edges and circuitless $\Rightarrow$ Connected.

Tree :- $T = \langle V, E \rangle$

$|V| = n \quad |E| = n-1$

$V = \{v_1, v_2, \ldots v_n\}$

$\deg(v_1) + \deg(v_2) + \ldots \deg(v_n) = 2n-2.$

- No vertex has zero degree.
- distribute 1 to every vertex.

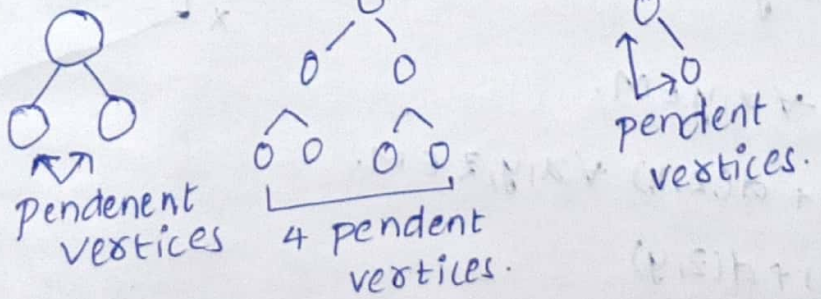
v_1	v_2			v_n
1	1	1	1	1
	1	1	1	1

$\deg(v_i) = 1$

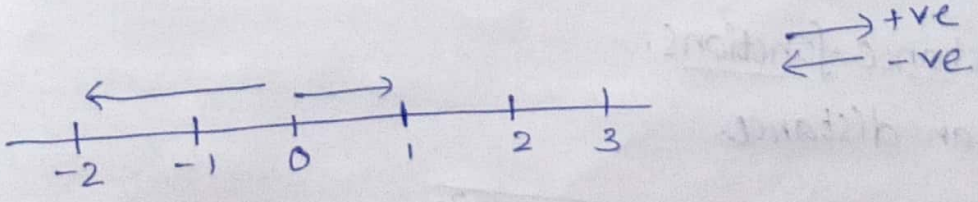
$n-2$

- 2 places would remain deg = 1 after destributing n-2 again.

- Any Tree will have atleast 2 pendent vertices.



- Numbers are distance from origin



Neighbour:

A closer one to me.

Relative to me.

$d(x,y) < 6$, is neighbour.

Place - Neighbourhood.

School - Areas

State capital - District.

Other state - State.

North of country - North v/s South

Other country - Is he from India?

US - Texas

Telugu community.

Sikh community.

latin community

mexian community.

distance:

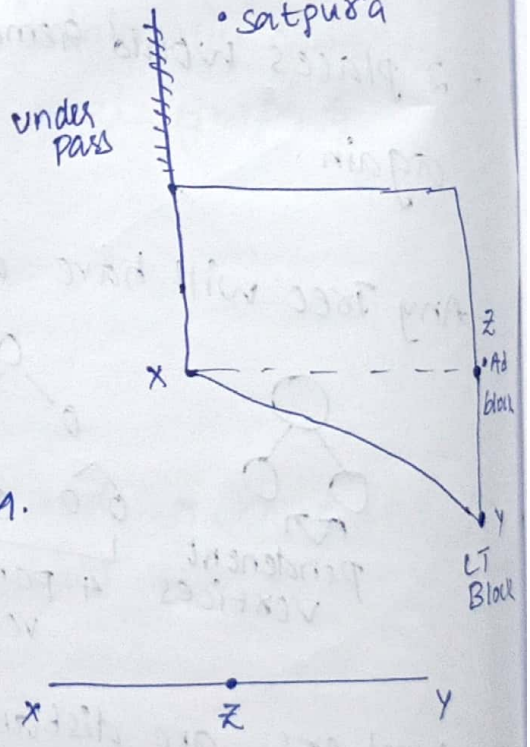
$d(a,b) = |b-a|$

A function, binary

$d: M \times M \longrightarrow \mathbb{R}$

M = Any set

{peoples, Numbers, Benchs, fruits, cities}



i) $d(x,y) \geq 0 \quad x,y \in M$

$d(x,y) = 0 \iff x = y$.

ii) $d(x,y) = d(y,x) \quad \forall x,y \in M$.

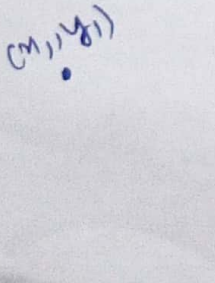
iii) $d(x,y) \leq d(x,z) + d(z,y) \quad \forall x,y,z \in M$.

if $d(x,y) = d(x,z) + d(z,y)$

they are collinear

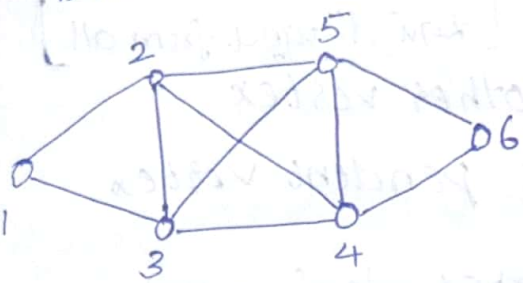
Multiple distance functions.

Eulicdian distance.



$$d = \sqrt{(x_2-x_1)^2 + (y_2-y_1)^2}$$

Distance b/w 2 vertices in graph:



G

dist b/w 1 & 2.

1 — 2

1 — 3 — 2

1 — 3 — 4 — 2

x 1 — 3 — 4 — 5 — 3 — 4 — 5 ....

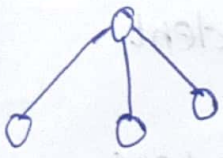
loop

Alternating path of vertices & edges where no vertex is repeated again.

len of path = No of edges in path.

$d(v_i, v_j)$ – length of shortest path b/w v_i & v_j (Graph)

$d(v_i, v_j)$ - length of path b/w v_i & v_j (Tree)



Only 1 path ovalable b/w v_i, v_j

dist b/w 1 & 5 in G

It should travel in less no of vertexes b/w them.

via set – minimum.

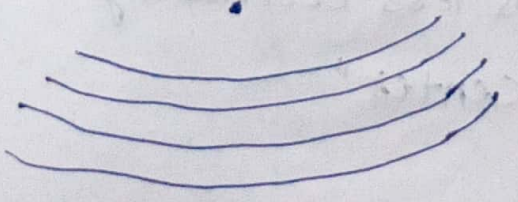
✓ 1 - 2 - 5] shortest paths.

✓ 1 - 3 - 5

1 - 3 - 2 - 5

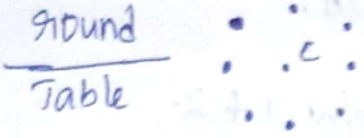
1 - 3 - 4 - 5

1 - 3 - 4 - 6 - 5



In real world, In order to have good audibilty of lecturer, he should stand in the middle of the class, to get good coverage.

Ground / Table

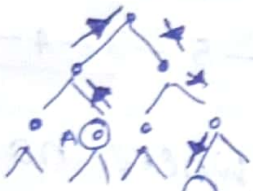


Real world Scienario.

[Center = Equi distance from all]

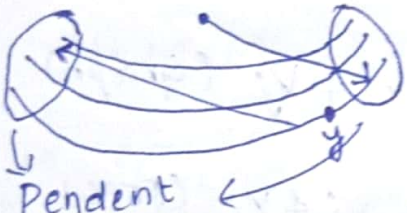
- Max distance of a vertex and any other vertex then the other vertex is always be pendent vertex

(pendent vertex = leaf nodes)



O max distance from A, pendent vertex

5/02/26.



Pendent vertecies.

Without any aids if a person wants to deliver a speech, where should he stand and deliver speech in order he auidable for everyone?

→ He/she should worry about pendent vertex

→ If he/she auidable to pendent vertexs he is auidable for everyone

class.

→ The person who sat on corner is the max distance to person who delivers speech

→ Eccentricity is nothing but the distance b/w them. (max possible distance in the graph)

→ for y there is not same eccentricity as the person on stage he has any other eccentricity.

→ which of the vertex has less eccentricity that vertex is called "Center"

$G = \langle V, E \rangle$

$E(v_i) = \max_{v_j \in V} d(v_i, v_j)$

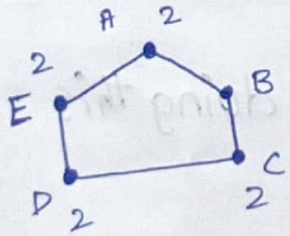
$centre(G) = \{ v_i \mid v_i = \min \Sigma(v_i) \}$

```
def bfs(graph, start):
    visited = set()
    distance = {node: float('inf') for node in graph}
    queue = deque([start])
    visited.add(start)
    distance[start] = 0
    while queue:
        node = queue.popleft()
        for neighbour in graph[node]:
            if neighbour not in visited:
                visited.add(neighbour)
                distance[neighbour] = distance[node]+1
                queue.append(neighbour)
    return distance
```

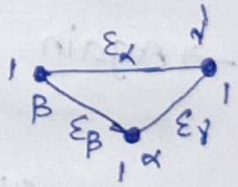
```
def find_eccentricity(graph):
    eccentricity = {}
    for vertex in graph:
        distance = bfs(graph, vertex)
        eccentricity[vertex] = max(distance.values())
    return eccentricity

def find_center(graph):
    ecc = find_eccentricity(graph)
    radius = min(ecc.values())
    center = [v for v in ecc if ecc[v]==radius]
```

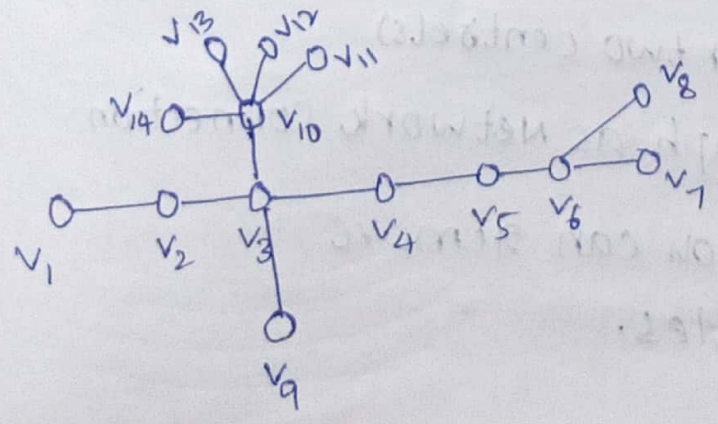
→ There may be one or many centre may exist



Here every vertex is centre.



→ How many centres possible on tree?



dist b/w 2 vertices

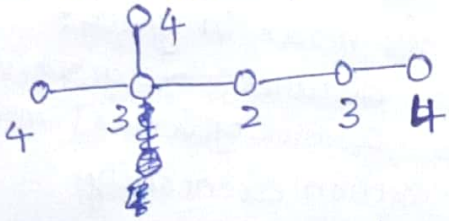
= No of edges b/w them

vertex	eccentricity	vertex	EC.	Ver	EC.
v_1	6	v_6	5	v_6	5
v_2	5	v_7	6	v_{11}	6
v_3	4	v_8	6	v_{12}	6
v_4	3	v_9	5	v_{13}	6
v_5	4			v_{14}	6

C

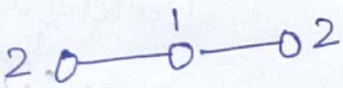
→ centre is v_4

→ delete all pendent vertexes



Eccentrity is reduced by one.

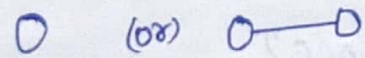
→ Again remove pendent vertexes



→ Again

v_4

→ you will ends up in 2 possibites while doing this



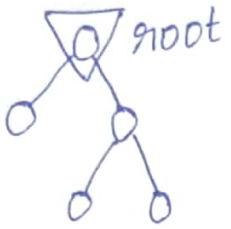
centre (or) bi-centred Tree.

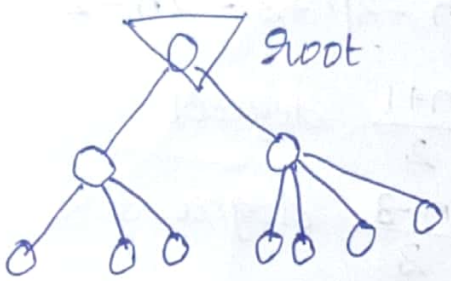
→ So tree can have one or two centre(s)

→ Assume the previous graph as Network connection

By destroying centre you can remove connection for max nodes.

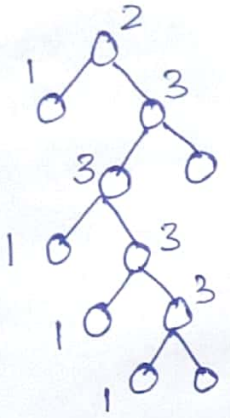
Rooted Tree:



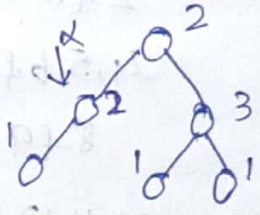


Binary Tree:

A special case of rooted tree, where every vertex can have ~~atmost~~ two outcomes/NO outcome.



Binary Tree



Not an binary tree
α has one outcome
It should have either 0/2 outcomes.

In binary tree, only root node has degree two
Every other vertex has either one/three as degree.

6/02/26

- In order to reduce Time complexity, we use binary tree
- No of vertices in binary Tree are odd.
- deg of v in binary Tree $\in \{1, 2, 3\}$

No of vertices of deg 1 = p (pendent nodes)
No of vertices of deg 3 = n − p − 1

$$p + 2 + 3(n - p - 1) = 2(n - 1)$$

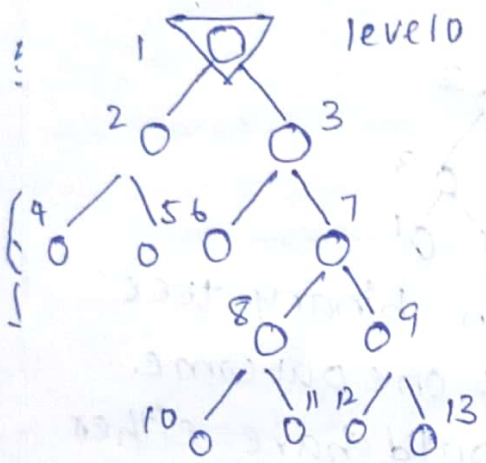
$$P + 2 + 3n - 3p - 3 = 2n - 2$$

$$p = \frac{n+1}{2} \quad \text{degree 1}$$

$$n - p - 1 = \frac{n-3}{2} \quad \text{degree 3.}$$

→ No of vertices of deg 1 = No of vertices of deg 3 + 1

Level



2 & 3 are level 1

4, 5, 6 & 7 are in level 2

8 & 9 are in level 3

10, 11, 12 & 13 are in level 4

fig : Tree T

- Max no of vertices at k level of B.T = 2^k

Max no of vertices upto k level of B.T = $2^0 + 2^1 + \cdots 2^k \geq n$

Path length.

Sum of length of all levels of pendent vertices

Ex for above Tree T path length = ~~8 + 4 = 6~~.

$= 2 \cdot 3 + 4 \cdot 4$

$= 6 + 16$

$= 22.$

$$\sum_{v_i, \deg(v_i)=1} l_i$$

Weighted path length.

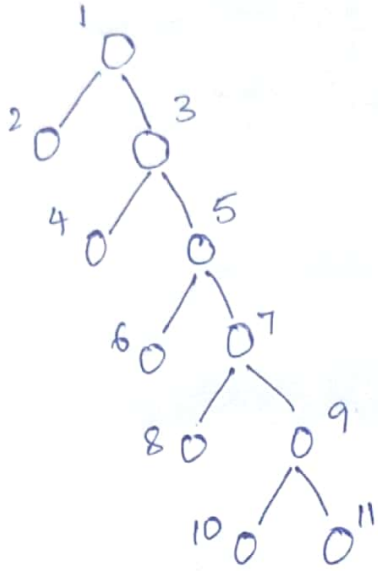
$$\sum_{v_i, \deg(v_i)=1} w_i l_i$$

for eg in Tree T

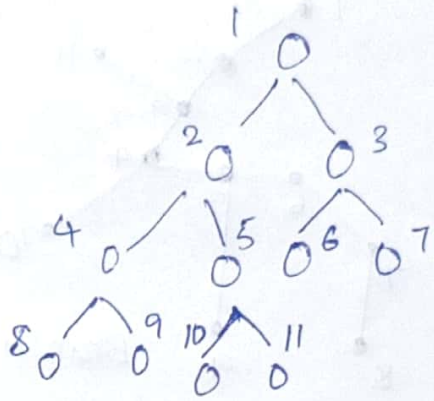
$w_1 = 0.2, w_2 = 0.3, w_3 = 0.4, w_4 = 0.3$

weighted path length = $2 \times 0.3 \times 3 + 4 \times 4 \times 0.3$

$= 6.6$



Worst case



Optimal case.

- No of trees are possible with n vertices =

no regular Formula need to draw with hands & count them.

9|Feb|26

The No of labeled Trees with n vertices is n^{n-2}

- A graph G with n vertices and edges. $G = \langle V, E \rangle$

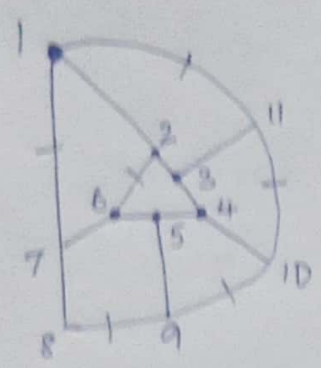
$|V| = n, |E| = e$

No of Subgraphs possible $= 2^e$

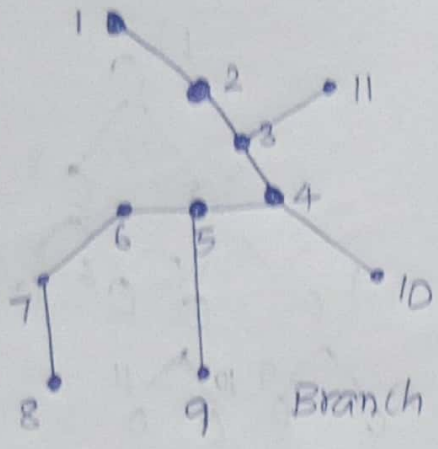
- Spanning Tree | Scaffolding | Skeleton:

- Maximal Sub tree.

Every connected graph a spanning tree exists.



Farmland



Branch

- The edges which are in:

Spanning tree - Branch B

not in spanning tree - Chord C

$|B| = n-1$

$|C| = e-n+1$

- A dissconnected graph with K components

$G = \langle V, E \rangle$

$|V| = n \quad (\geq k)$ $\qquad |E| = e$

$n_1 + n_2 + n_3 + \dots n_k = n$ $\qquad e_1 + e_2 + \dots e_k = e$

$e_i \geq n_i - 1$ — summation on both sides.

$e \geq n-k$

$e-n+k \geq 0$

Rank of $G = n-k \quad (\geq 0)$ } Branch } in connected graph

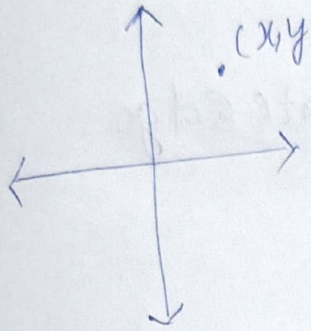
Nullity of $G = e-n+k \quad (\geq 0)$ } Chord } $(k=1)$

Spanning Forest.

Note:

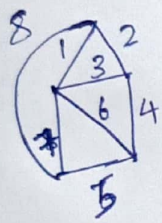
Cyclomatic Number / Beti Number $= e-n+k$.

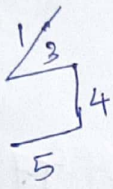
- Adding a edge between 2 vertices in a tree a circut will be created. We will called it "fundamental circuts"



span of $\{(0,1)\}$ $\Rightarrow$ $x=0$.

12/02/26





Tree

fundamental circut

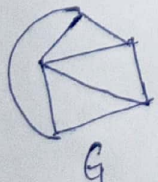
- Which can you find easily?

No of circuts.
No of fundamental circuts.

Why? Justify?

- Whats the difference b/w spanning tree & Hamiltonian path?

Hamiltonian path: No vertex is not repeated again



G

Hamiltonian path & spanning tree

Spanning tree but not Hamiltonian path.

- A spanning tree which has degree of all vertiles is 2 except the terminals is Hamiltonian path.

- Standard way to construct a Spanning tree.

1. Start from any random Spanning tree

T_1: 

2. Add chord to T1 & remove appropriate Edge for Spanning Tree.

T_2: 

T_3: 

Repeat step2.

T_4: 

T_5: 

T_6: 

T_7: 

By doing this* we can construct all graphs / spanning trees. *repeatedly.

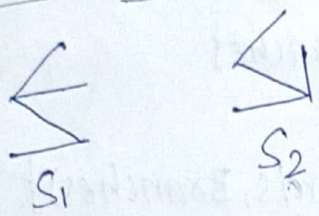
- This Method is Cyclic interchange / Elementary Tree Transformation.

- No of fundamental circuits = No of chords in a graph.

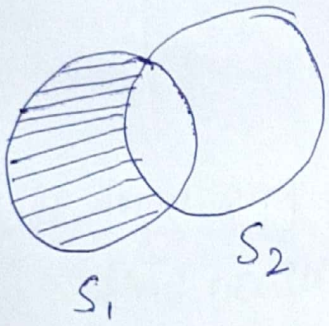
$= E - N + 1$

Central Tree:

Distance b/w Spanning Tree



1. distance b/w S_1 & S_2 = edges are in S_1 which are not in S_2.



$= 1$

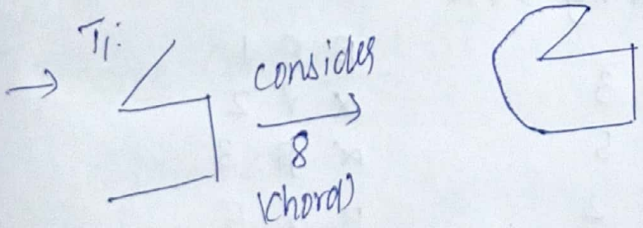
2. Set of edges in $S_1 = e_1$
Set of edges in $S_2 = e_2$

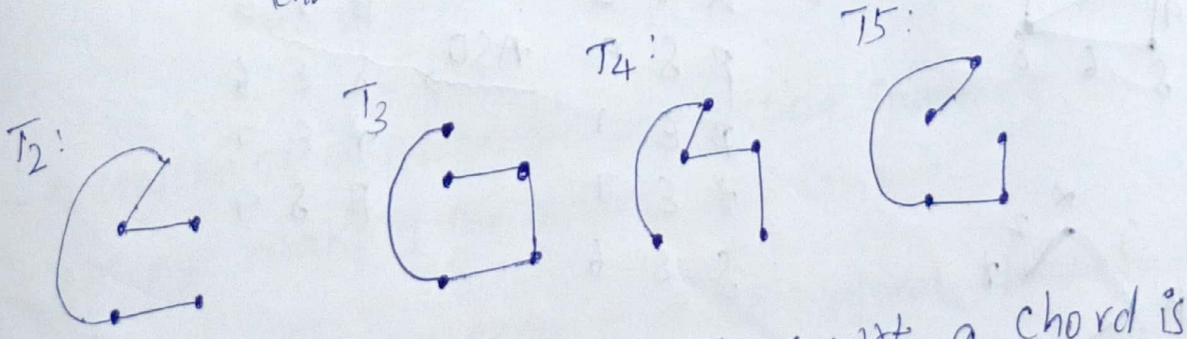
distances b/w Spanning trees $= |e_1 - e_2|$

3. $d(T_1, T_2) = \frac{N(T_1 \oplus T_2)}{2}$

$\oplus$ Ring Sum.

$\max\{d(T_0, T_j)\} \leq \max\{d(T_i, T_j)\}$





distances b/w any 2 spanning Trees wot a chord is always one.

→ $d(T_1, T_2)$ cannot exceed no of chords.

$d(T_1, T_2)$ cannot exceed no of branches

$d(T_1, T_2)$ cannot exceed $\#\{\min(\text{chords}, \text{Branches})\}$

$d(T_1, T_2) \leq \min(\#\text{chords}, \#\text{Branches})$

$d(T_1, T_2) \leq \min(\text{rank}, \text{Nullity})$

Central tree:

$d(T_0, T_1) \leq \min\{d(T_i, T_j)\}$

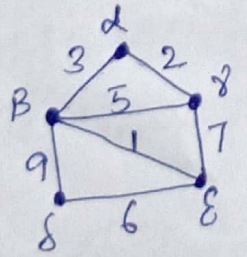
- Wath's common b/w all spanning trees:

All spanning trees has same no of edges.

$= n - 1$.

Kruskal's Algorithm.

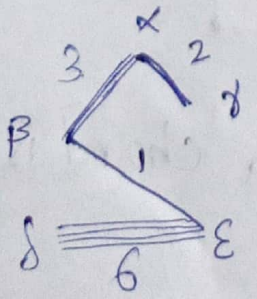
Minimum weighted Spanning Tree.



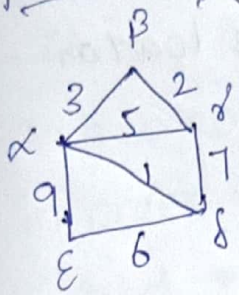
α	β	3
β	γ	5
α	γ	2
β	δ	9
β	ε	1
γ	ε	7
δ	ε	6

ASO →

β	ε	1
α	γ	2
α	β	3
β	γ	5
δ	ε	6
γ	ε	7
β	δ	9



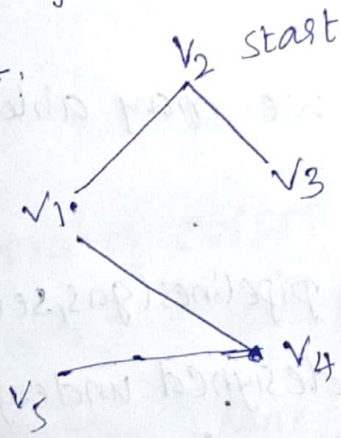
Prims Algorithm



∞	3	5	1	9
3	∞	2	∞	∞
5	2	∞	7	∞
1	∞	7	∞	6
9	∞	∞	6	∞

start from any row
- find nearest neighbour

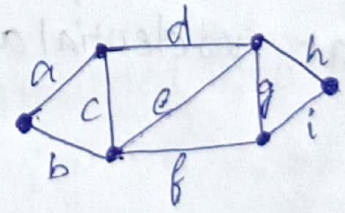
MST:



13/02/26

Degree Constraint ST:-

$deg(v_i) \leq K$
└ each v_i in ST



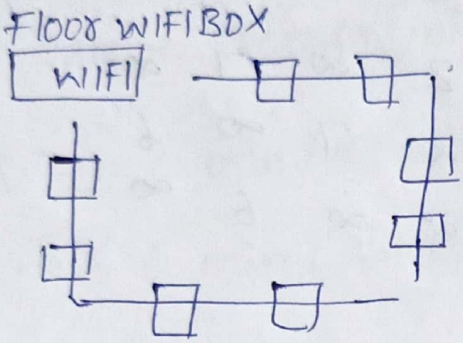
- A constraint is added for constructing spanning tree we are making the degree constraint.

deg of every edge $\leq$ K (A Natural Number)

Q) Do you think it is useful?

1. A fuse cannot have all the load of morcleon house. & we divide the load for multiple ones, applying the degree constraint.

2. In wifi network, if we need to give multiple users (like BH-2) we use many switches to reduce the load on single controller.



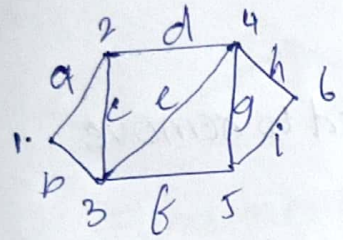
The topology of routers/switches are designed by No of users & speed connectivity.

3. Making spanning tree $deg(v_i) \leq 2$, we may able to get Hamiltonian graph.

4. In planar graphs, let use think city pipelines (gas, sewage, water, even metros nowadays) are designed underground and at an junction they calluculatly divide into some connection at residential areas.

CUTSETS.

- It is set of cuts.
- In a connected graph G, we remove the edges of graph in order to make graph disconnected in minimal way



remove a, b.

graph disconnected.

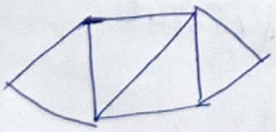
{a, b}

remove h, i — graph disconnected

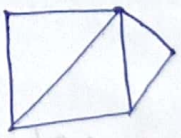
{h, i}

- There are many CUT-SETS. available.

- Removal of CUTSET from a given connected graph G reduces the rank(N-K) by one exactly.



Rank = 6 - 1
= 5



Rank = 6 - 2
= 4

- A set of vertices can be partitioned into two, remove the edges which are all joining V_1 & V_2:

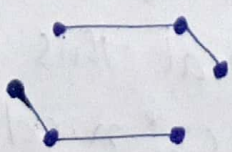
for ex: {1, 2, 3, 4, 5, 6}

$V_1 \cup V_2 = V$

$V_1 \cap V_2 = \phi$

$V_1 = \{1, 3, 5\}$ $V_2 = \{2, 4, 6\}$

Now remove all edges connecting V_1 & V_2



CUTSET = {a, c, e, g, i}


ST:

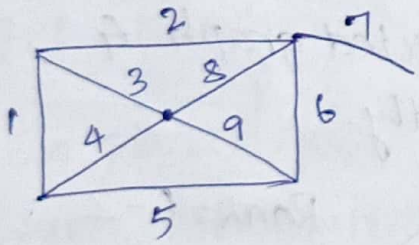

- Every spanning tree has one edge which should be removed in order to make graph dissconnected.

- circuts & cutsets.

In order to obtain cutset from circut, we need to remove ... atleast 2 edges.

- No of edges are in common b/w any circut & cutset

Theorm: Every cutset will have even no of edges in common with any circut.

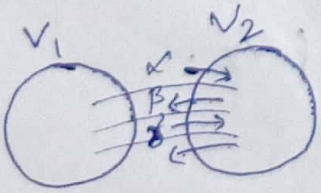


2 {1, 3, 2} {7}
4 {2, 8, 9}
2 {1, 4, 5}
{2, 8, 6}

proof: Graph G is connected, S is an CUTSET, 7 is circut.

7 is lies in entirely in V_1 | V_2

No of common edges = 0.



7 is lies in V_1 & V_2.

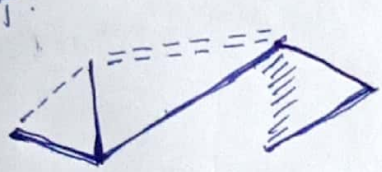
When we start from, V_1 to travel to V_2 in order to complete circut we need to travel back to V_1 (2 edges) if circut is not completed, we repeat this process until a circut formed (2* n edges

- #(CUT-SETS)?

A connected graph G can have many CUTSETS.

We study about Fundamental CUTSETS.

ST:



Fundamental circut.

1 path → 2 paths.
only one chord

Fundamental CUT-SET

0 paths → 1 path.
only one branch.

- A Fundamental CUT-SET is which contains only one branch from Spanning Tree

#(Fundamental CUTSET) = No of branches.

16/02/26

Spanning Tree of G

Fundamental circut

Add a chord

Fundamental CUTSET

Remove one Branch

- Spanning Tree is Minimally connected.

Removing any one of edge leads to disconnected graph.

Edge Connectivity & vertex Connectivity.

wrt ST, a chord that determines the Fundamental circuit (T) occurs in every fundamental cutset associated with each branch of T and in no other.

proof:

let T be S.T &

$T = \{b_1, b_2, \ldots, b_{n-1}\}$

$C = \{c_1, c_2, \ldots c_k\}$

c_i add it T $\Rightarrow T = \{c_i, b_1, b_2 \ldots br\}$

b_1 add it to C $\Rightarrow S = \{b_1, c_1, c_2, \ldots c_q\}$

b_2 add it to C $\Rightarrow S_2 = \{b_2, c_1', c_2' \ldots c_z'\}$

Suppose if $\exists$ cut-set S_{r+1} that contains b_{r+1} & T_{r+1}

→wrt St, a branch that determines the fundamental CUTSET occurs in every Fundamental circut associated with chord of s and in no other.

proof:

let T be some ST

$T = \{b_1, b_2, \ldots b_{n-1}\}$

$C = \{c_1, c_2, \ldots c_k\}$

Add b_i to C $\Rightarrow S = \{b_i, c_1, c_2, \ldots c_r\}$

Add c_1 to T $\Rightarrow T = \{c_1, b_1, b_2, \ldots b_m\}$

Add c_2 to T $\Rightarrow T = \{c_2, b_1', b_2', \ldots b_y'\}$

Suppose if $\exists$ fundamental circut T_{r+1} that contains c_{r+1} & S_{r+1}.